

Automated Logical Deciders

Certificate-Indexed Halting, Multiagent Search, Provenance-Aware Shared Lemma Pools, and Machine-Learned Guidance

A rigorous research supplement in the spirit of
Depths of Induction, ML-Guided Proof Search, and Formalized Self-Awareness,
Version 47

Brian Tenneson

M.S., Applied Data Science

Currently Unaffiliated

August 2, 2026

This supplement develops Automated Logical Deciders (ALDs) as certificate-producing, multiagent proof-search systems. Four principal roles search in parallel for a proof of a conjecture, a proof of its negation, a contradiction, and metatheoretic certificates that each side is underivable. Every agent can read from and propose additions to a shared lemma pool in every round. Pool entries are independently verified, typed by object or metatheory, and labeled by the assumptions on which they depend. The central principle is that operational termination, exhaustive closure, and logical settlement are distinct. A conclusive status is licensed only by a status-appropriate certificate.

Abstract

An ordinary target-search automated theorem prover is naturally success-oriented: it halts when it finds a proof. That architecture cannot detect independence merely by running a prover for a conjecture and another for its negation, because when both conjectures are underivable both searches may run forever. This paper replaces failure-to-halt with positive, independently checkable certificates of nonderivability. It defines a multiagent Automated Logical Decider with four principal roles: a proponent for c , an opponent for $\neg c$, an inconsistency agent for $\perp$ or $c \wedge \neg c$, and an independence agent searching in a declared metatheory for two nonderivability certificates. All agents can read from and contribute to a provenance-aware shared lemma pool in every round; only verifier-accepted entries are published, and every entry records the assumptions under which it is valid. Sufficient certificates include finite exhaustive saturation, preserved invariants, model pairs, relative-consistency arguments, and arbitrary checked metaproofs. The paper proves soundness, context-safe and conservative lemma reuse, certificate-horizon results, fair multiagent halting under certificate coverage, a no-false-independence theorem for timeouts, and undecidability barriers to total ALDs. It then adds machine learning as a soundness-neutral policy for ranking actions, allocating resources, and selecting reusable lemmas, provided independent verification and a fairness-preserving exploration component remain fixed. A literature comparison concludes that the ingredients have substantial prior art—including the TPTP SZS *NoConsequence* status, cooperative theorem proving, blackboard architectures, model finding, lemma reuse, and formal independence proofs—while the specific certificate-indexed SIC-style synthesis with a universally visible, assumption-labeled lemma pool appears to be a plausible original research direction rather than an established theorem of novelty.

Contents

Abstract	i
1 Orientation: from proof search to logical settlement	1
1.1 The original obstruction	1
1.2 Three distinct notions of stopping	1
1.3 Relation to existing automated reasoning practice	1
2 Prior art and originality assessment	2
2.1 What is already established	2
2.2 The candidate-original synthesis	2
3 Formal setting	3
3.1 Object theory and metatheory	3
3.2 Logical evidence and status claims	3
4 The round-synchronous multiagent ALD	4
4.1 Principal agents	4
4.2 Provenance-aware typed shared lemma pool	4
4.3 Global state	5

5	Soundness and conservativity of the shared pool	6
6	Sound status extraction	7
7	Positive certificates of nonderivability	8
7.1	Finite saturation certificates	8
7.2	Invariant certificates	9
7.3	Model certificates	9
7.4	Relative-consistency certificates	10
7.5	General metaproof certificates	11
8	Certificate horizons and ALD halting	11
8.1	Minimal certificate length	11
8.2	Fair multiagent coverage	11
8.3	Unknown safety	12
8.4	Undecidability barriers	12
9	How the four agents help one another	13
9.1	Cross-role contributions	13
9.2	A blackboard interpretation	14
10	Machine-learned guidance as a later layer	14
10.1	What learning may control	14
10.2	Pool-aware learning	15
11	Examples of ALD settlement	16
11.1	Finite propositional logic	16
11.2	Finite algebra and first-order theories	16
11.3	Set-theoretic independence	16
11.4	Predator-style ALD	16
12	Further theory	17
12.1	Settlement depth vectors	17
12.2	Certificate transport under recoding	17
12.3	Settlement as a partially ordered evidence process	17
12.4	Research conjectures	18
13	Implementation blueprint	18
13.1	Required audit record	19
14	Limitations and boundaries	19

15 Conclusion	20
References and research context	21

1 Orientation: from proof search to logical settlement

1.1 The original obstruction

Suppose a theory $\mathcal{T}$ and sentence c are fixed. One ATP searches for $\mathcal{T} \vdash c$ and another searches for $\mathcal{T} \vdash \neg c$. Parallelism helps when one side is derivable, but it does not solve independence. If neither side is derivable, the two searches may simply fail to halt. Their joint silence is not a certificate: it may mean independence, insufficient resources, a poor heuristic, an implementation fault, or merely that the shortest proof has not yet been found.

The key replacement is:

absence of proof search success $\longrightarrow$ presence of a checked nonderivability certificate.

An independence agent is therefore not merely a third object-theory ATP. It is a certificate-search procedure operating in a declared metatheory $\mathcal{M}$ that can verify claims about derivability in $\mathcal{T}$.

1.2 Three distinct notions of stopping

Definition 1.1 (Operational stopping, exhaustive stopping, and settlement). Let a computational proof-search system run on $(\mathcal{T}, c)$.

- (i) It *stops operationally* when its program ceases execution for any reason.
- (ii) It *stops exhaustively* when it supplies a certificate that a declared search domain or certificate class has been completely explored.
- (iii) It *settles* a logical claim when it supplies a certificate accepted by a sound verifier for that claim.

A timeout is operational stopping. Finite saturation can be exhaustive stopping. A proof, model pair, invariant argument, or checked metaproof can be logical settlement. These categories must not be conflated.

1.3 Relation to existing automated reasoning practice

The architecture proposed here has antecedents, but the synthesis is specific. Distributed deduction has long allowed concurrent processes to exchange clauses and divide search [2, 3, 4]. Leo-II cooperates with specialist first-order provers, and Leo-III was explicitly designed around a multiagent blackboard architecture [5, 6]. Agent-based first-order proving has also been studied with restricted inter-agent languages [7]. Large-theory ATP work mines and reuses enormous lemma collections, often with learned relevance filtering [12, 13]; ENIGMA learns clause-selection guidance [14]. Recent LLM-agent systems likewise generate auxiliary lemmas or distribute formalization work among agents while relying on a proof assistant as checker [15, 16]. Search/check separation itself is classical [8].

The present theory does not claim that cooperation, model finding, proof certificates, lemma reuse, or machine learning are individually new. It asks a different organizing question: what finite evidence licenses a multiagent system to report that a conjecture is proved, negation-proved, inconsistent, independent, or merely unknown, and how may every role safely share intermediate results at every round?

2 Prior art and originality assessment

2.1 What is already established

The closest existing status vocabulary is the TPTP SZS ontology. For an axiom set T and conjecture c , its *NoConsequence* status is justified by a pair of models or saturations, one satisfying $T \cup \{c\}$ and one satisfying $T \cup \{\neg c\}$ [1]. Under soundness and completeness, that is exactly the semantic form of syntactic independence used below. Thus the idea that independence can be positively witnessed by two models is not new.

Likewise, each of the following is established prior art:

Ingredient	Representative prior work
Concurrent and distributed proof search	Clause-Diffusion, Aquarius, TEAMWORK, DISCOUNT, and requirement-based cooperative proving exchange clauses or facts and divide proof search [2, 3, 4].
Multiagent blackboards	Leo-III was proposed with reasoning agents cooperating through a blackboard at term, clause, and search levels [6].
Proof plus model search	The TPTP/SZS framework distinguishes theorem, counter-satisfiable, contradictory-axiom, satisfiable, and no-success outcomes; finite model finders provide checkable countermodels [1, 9, 10].
Formal independence proofs	Deep metamathematical independence arguments can be formalized and kernel-checked; the Lean formalization of the independence of CH is a direct example [11].
Reusable lemma pools	Lemma mining, lemmatization, and learned relevance filtering strengthen large-theory proving [12, 13].
Learned proof guidance	ENIGMA and later prover agents use learned ranking, retrieval, or auxiliary lemmas while a formal checker determines acceptance [14, 15, 16].

2.2 The candidate-original synthesis

The potentially original contribution is narrower and should be stated conservatively. In the literature reviewed for this supplement, no exact prior formulation was located that simultaneously provides:

- (i) a SIC-style formal object whose terminal outcomes are indexed by proof, negation-proof, contradiction, and paired nonderivability certificates;
- (ii) a theorem-level separation of operational stopping, certified exhaustive closure, and logical settlement;
- (iii) a universally visible read-write lemma pool updated every round, with independent checking, object/meta typing, and explicit assumption provenance;
- (iv) soundness, support-discharge, fair-coverage, certificate-horizon, timeout-safety, and undecidability theorems for that architecture; and
- (v) a verifier-separated machine-learning layer that may guide every role and lemma retrieval without changing accepted logical status.

This is evidence for a plausible original line of research, not a proof of worldwide novelty. A definitive originality claim would require a broader systematic review, expert comparison, and possibly peer review. The strongest defensible claim at present is: *the ingredients are known, while their certificate-indexed ALD synthesis and provenance-aware shared-pool theory appear not to be standard in the sources examined.*

3 Formal setting

3.1 Object theory and metatheory

Fix a formal system $\mathcal{F} = (x, y, z)$ in the sense of Version 47, a hypothesis set $\Gamma \subseteq y$, and a conjecture $c \in y$. Assume the language contains a negation operation on the formulas under discussion and a contradiction formula $\perp$. Write

$$\Gamma \vdash c, \quad \Gamma \vdash \neg c, \quad \Gamma \vdash \perp$$

for derivability in $\mathcal{F}$. Let $\mathcal{M}$ be a declared metatheory capable of representing finite certificates, verifier computations, and the needed soundness claims about $\mathcal{F}$.

Remark 3.1 (Relativity). Every independence conclusion in this paper is relative to the object theory $\mathcal{F}$, the hypotheses Γ , the proof relation, and the metatheory $\mathcal{M}$ used to certify nonderivability. An ALD does not produce an unqualified metaphysical fact called “independent.” It produces a checked statement of the form

$$\mathcal{M} \vdash (\Gamma \not\vdash_{\mathcal{F}} c) \wedge (\Gamma \not\vdash_{\mathcal{F}} \neg c).$$

3.2 Logical evidence and status claims

Definition 3.2 (Basic evidence claims). For the instance $(\mathcal{F}, \Gamma, c)$ define four basic evidence claims:

$$\begin{aligned} \mathbf{P} &: \Gamma \vdash c, \\ \mathbf{N} &: \Gamma \vdash \neg c, \\ \mathbf{B} &: \Gamma \vdash \perp, \\ \mathbf{I} &: (\Gamma \not\vdash c) \wedge (\Gamma \not\vdash \neg c). \end{aligned}$$

The first three are object-theoretic derivability claims. The fourth is a metatheoretic conjunction of nonderivability claims.

Definition 3.3 (Certificate scheme). A *status-indexed certificate scheme* consists, for every $e \in \{\mathbf{P}, \mathbf{N}, \mathbf{B}, \mathbf{I}\}$, of a set Cert_e of finite certificate strings and a total decidable verifier

$$\text{Ver}_e(\mathcal{F}, \Gamma, c, \pi) \in \{0, 1\}.$$

The scheme is *sound in $\mathcal{M}$* if, for every e and $\pi \in \text{Cert}_e$,

$$\text{Ver}_e(\mathcal{F}, \Gamma, c, \pi) = 1 \implies \mathcal{M} \vdash \text{Claim}_e(\mathcal{F}, \Gamma, c).$$

Definition 3.4 (Evidence ledger). An evidence ledger is a subset $E \subseteq \{\mathbf{P}, \mathbf{N}, \mathbf{B}, \mathbf{I}\}$ containing exactly those evidence types for which the global verifier has accepted at least one certificate.

Definition 3.5 (Derived status). Define the display status $\text{Status}(E)$ by the following priority:

$$\text{Status}(E) = \begin{cases} \text{INCONSISTENT}, & B \in E, \\ \text{BOTH}, & P, N \in E, \\ \text{INDEPENDENT}, & I \in E, \\ \text{DERIVABLE}, & P \in E, N \notin E, \\ \text{REFUTABLE}, & N \in E, P \notin E, \\ \text{UNKNOWN}, & \text{otherwise.} \end{cases}$$

The labels DERIVABLE and REFUTABLE assert derivability and derivability of the negation, not semantic truth or falsity. This makes them valid even before the consistency of Γ is established.

4 The round-synchronous multiagent ALD

4.1 Principal agents

Definition 4.1 (Principal ALD agents). A four-role ALD contains:

- (1) a *proponent* A_+ searching for a P certificate;
- (2) an *opponent* A_- searching for a N certificate;
- (3) an *inconsistency agent* $A_\perp$ searching for a B certificate, or for proofs of both c and $\neg c$;
- (4) an *independence agent* A_{ind} searching in $\mathcal{M}$ for an I certificate.

Additional specialist agents may search for models, invariants, finite closures, translations, proof lower bounds, or useful intermediate lemmas, but their accepted outputs are routed to one of the four evidence classes or to the lemma pool.

4.2 Provenance-aware typed shared lemma pool

A single untyped pool is dangerous for two independent reasons. First, a theorem of the metatheory must not silently become an axiom of the object theory. Second, a lemma proved only under the temporary assumption c must not be reused as though it followed from Γ alone. The pool therefore records both *logical level* and *assumption support*.

Definition 4.2 (Object-pool record). An object-pool record is a quadruple

$$(\text{obj}, \Delta, \ell, \pi),$$

where $\Delta \subseteq y$ is finite, $\ell \in y$, and π is an accepted certificate of

$$\Gamma \cup \Delta \vdash_{\mathcal{F}} \ell.$$

The finite set Δ is the *support* or *provenance context* of the record, written $\text{supp}(\ell, \pi) = \Delta$.

Definition 4.3 (Metapool record). A metapool record is a triple

$$(\text{meta}, \mu, \rho),$$

where μ is a metatheoretic statement and ρ is an accepted $\mathcal{M}$ -certificate of μ . If μ concerns an object-level branch, the encoded branch assumptions are part of the statement itself.

Definition 4.4 (Admissible use). An object agent currently reasoning from $\Gamma \cup \Theta$ may use a record $(\text{obj}, \Delta, \ell, \pi)$ when it has accepted certificates of

$$\Gamma \cup \Theta \vdash \delta \quad \text{for every } \delta \in \Delta.$$

The syntactic condition $\Delta \subseteq \Theta$ is a simple sufficient case. A base lemma has $\Delta = \emptyset$ and is admissible to every object-level branch.

At round r , the global pool is

$$\mathbb{L}_r = \mathcal{L}_r^{\mathcal{F}} \sqcup \mathcal{K}_r^{\mathcal{M}},$$

where $\mathcal{L}_r^{\mathcal{F}}$ contains accepted object-pool records and $\mathcal{K}_r^{\mathcal{M}}$ contains accepted metapool records. Every agent can inspect both components in every round. Use is governed by level and support admissibility, so universal visibility does not mean indiscriminate import.

Protocol 4.5 (Round protocol with universal read-write access). At round r :

- R1. Read.** Every agent receives the full snapshot $\mathbb{L}_r$, its own current assumption context, and the evidence ledger E_r .
- R2. Search.** Each agent performs its allotted work using private state and any pool records admissible in its current context.
- R3. Propose.** Every agent may submit candidate object lemmas with explicit support, metatheoretic facts, support-discharge requests, and status certificates.
- R4. Verify.** Independent checkers validate the claimed derivation, logical level, support, and certificate type.
- R5. Publish.** All accepted records are appended to the pool and all accepted status certificates are appended to the evidence ledger:

$$\mathbb{L}_{r+1} := \mathbb{L}_r \cup \text{AcceptedRecords}_r, \quad E_{r+1} := E_r \cup \text{AcceptedEvidence}_r.$$

Thus every agent can add to and utilize the lemma pool in every round, while the verifier prevents object/meta confusion and assumption leakage. An asynchronous implementation may broadcast an accepted record immediately; the round model supplies a deterministic audit semantics.

4.3 Global state

Definition 4.6 (ALD global state). Let $Q_{i,r}$ be the private state of agent i at round r . The global ALD state is

$$G_r = ((Q_{i,r})_{i \in I}, \mathbb{L}_r, E_r, r).$$

A global halting policy may halt on any accepted evidence type or may continue after DERIVABLE or REFUTABLE in order to test for BOTH or INCONSISTENT. The policy must state which statuses are terminal before the run begins.

5 Soundness and conservativity of the shared pool

Theorem 5.1 (Pool monotonicity and universal availability). *For all rounds r ,*

$$\mathcal{L}_r^{\mathcal{F}} \subseteq \mathcal{L}_{r+1}^{\mathcal{F}}, \quad \mathcal{K}_r^{\mathcal{M}} \subseteq \mathcal{K}_{r+1}^{\mathcal{M}}, \quad E_r \subseteq E_{r+1}.$$

Every record accepted in round r is visible to every agent at the read phase of round $r + 1$.

Proof. This is immediate from the union updates in the round protocol and the universal broadcast rule. $\square$

Theorem 5.2 (Provenance soundness). *Assume the object-record verifier is sound. Then every record $(\text{obj}, \Delta, \ell, \pi) \in \mathcal{L}_r^{\mathcal{F}}$ satisfies*

$$\Gamma \cup \Delta \vdash_{\mathcal{F}} \ell$$

for every round r .

Proof. Induct on r . Every initial record is required to carry an accepted certificate of its supported sequent. At the successor step, each new record is admitted only after the verifier accepts exactly that sequent. Old records remain sound by the induction hypothesis. $\square$

Theorem 5.3 (Metapool soundness). *Assume the metatheoretic verifier is sound in $\mathcal{M}$. Then every fact in $\mathcal{K}_r^{\mathcal{M}}$ is a theorem of $\mathcal{M}$ for every r .*

Proof. The same induction applies, using the metatheoretic verifier. $\square$

Theorem 5.4 (Context-admissible reuse). *Suppose $(\text{obj}, \Delta, \ell, \pi)$ is a sound pool record and an agent working from $\Gamma \cup \Theta$ has proofs of every $\delta \in \Delta$. Then*

$$\Gamma \cup \Theta \vdash_{\mathcal{F}} \ell.$$

Proof. The record supplies a finite proof of ℓ from $\Gamma \cup \Delta$. Replace every use of $\delta \in \Delta$ by its proof from $\Gamma \cup \Theta$. Finite proof splicing yields a proof of ℓ from $\Gamma \cup \Theta$. $\square$

Corollary 5.5 (Base-lemma universality). *Every record with empty support is admissible to every object-level agent and branch.*

Theorem 5.6 (Support discharge and promotion). *Suppose the pool contains*

$$(\text{obj}, \Delta \cup \{\alpha\}, \ell, \pi) \quad \text{and} \quad (\text{obj}, \Delta, \alpha, \sigma).$$

Then the verifier can construct and accept a promoted record

$$(\text{obj}, \Delta, \ell, \tau).$$

Proof. The first record proves ℓ from $\Gamma \cup \Delta \cup \{\alpha\}$ and the second proves α from $\Gamma \cup \Delta$. Substitute the second proof for the temporary assumption α in the first. The resulting finite derivation proves ℓ from $\Gamma \cup \Delta$. $\square$

Remark 5.7 (Why promotion matters). A branch agent may discover a powerful lemma under the hypothesis c . Initially that lemma is available only to agents whose context establishes c . If the proponent later proves c from the base theory, support discharge promotes the branch lemma to a base lemma, after which every object-level role can use it. Shared information can therefore become more widely reusable as evidence accumulates, without ever discarding provenance.

Theorem 5.8 (Conservative base-lemma reuse). *Let $L \subseteq \mathcal{Y}$ satisfy $\Gamma \vdash \ell$ for every $\ell \in L$. Then adjoining L as reusable derived lemmas does not change the consequence set:*

$$\text{Con}_{\mathcal{F}}(\Gamma \cup L) = \text{Con}_{\mathcal{F}}(\Gamma).$$

Proof. Monotonicity gives one inclusion. Conversely, replace each use of a member of L in a finite proof from $\Gamma \cup L$ by its finite proof from Γ . $\square$

Corollary 5.9 (Cross-agent lemma safety). *A base-level object lemma discovered by any agent may be used by every other object-level agent without changing theoremhood. A supported branch lemma may be used by every agent whose context discharges its support.*

Theorem 5.10 (Type-safety barrier). *Suppose object-level inference rules may consume only admissible records from $\mathcal{L}_r^{\mathcal{F}}$ and metatheoretic inference rules may consume records from $\mathcal{K}_r^{\mathcal{M}}$. Then a metatheoretic fact cannot strengthen the object theory unless an explicit, verifier-checked reflection or import rule licenses the move.*

Proof. By construction, the two record classes are disjoint and object-level rule applications cannot treat a metapool statement as an object formula. The only crossing is through a declared checked interface. $\square$

Corollary 5.11 (No branch-assumption leakage). *A theorem proved only from $\Gamma \cup \{c\}$ cannot be used as a theorem of Γ unless the support $\{c\}$ is discharged by a proof of c from Γ or by another checked rule.*

Proposition 5.12 (Search-space inclusion). *Fix the same agents and local inference mechanisms. Every derivation available when agents keep private pools is also available when all verified records are shared, because each agent may ignore additional records. The shared architecture therefore weakly enlarges the space of legal verified derivations.*

Theorem 5.13 (Conditional non-delay under extension-monotone scheduling). *Assume a scheduler is extension-monotone: adding legal actions never removes or postpones any action that would have been scheduled in the private-pool run. Then every certificate found by round r in the private-pool system is found by round r in the shared-pool system. Shared records may strictly accelerate settlement but cannot delay it under this hypothesis.*

Proof. Induct on rounds. The shared system contains every private transition and may contain additional verified transitions. Extension-monotonicity preserves the private actions at their original rounds. $\square$

Remark 5.14. Real resource-limited implementations may violate extension-monotonicity because extra records consume memory or alter priorities. The theorem isolates the ideal logical effect; engineering must control pool pollution, duplication, provenance checking, and relevance.

6 Sound status extraction

Theorem 6.1 (Evidence soundness). *Assume the status-indexed certificate scheme is sound in $\mathcal{M}$. Then every evidence bit in E_r denotes a correct claim in $\mathcal{M}$.*

Proof. Every evidence bit is added only after its verifier accepts a certificate. Soundness of the corresponding verifier yields the claim. $\square$

Theorem 6.2 (Status soundness). *Under the same hypothesis:*

- (i) $\text{Status}(E_r) = \text{DERIVABLE}$ implies $\Gamma \vdash c$;
- (ii) $\text{Status}(E_r) = \text{REFUTABLE}$ implies $\Gamma \vdash \neg c$;
- (iii) $\text{Status}(E_r) = \text{BOTH}$ implies both derivabilities;
- (iv) $\text{Status}(E_r) = \text{INCONSISTENT}$ implies $\Gamma \vdash \perp$;
- (v) $\text{Status}(E_r) = \text{INDEPENDENT}$ implies $\Gamma \not\vdash c$ and $\Gamma \not\vdash \neg c$.

Proof. Unwind Definition 2.5 and apply Theorem 5.1 to the evidence bits required by each case. $\square$

Proposition 6.3 (Contradiction from dual proof). *Assume $\mathcal{F}$ has a rule deriving $\perp$ from c and $\neg c$. If $P, N \in E_r$, then $\Gamma \vdash \perp$. Thus BOTH is a syntactic inconsistency witness even before the dedicated inconsistency agent succeeds.*

Proof. Concatenate the accepted proofs of c and $\neg c$ and apply the contradiction rule. $\square$

Remark 6.4 (Why retain a separate inconsistency agent). The inconsistency agent is not logically necessary once both side proofs exist, but it may find a much shorter route to $\perp$ than independently proving the selected c and $\neg c$. It also detects global inconsistency that is unrelated to the current conjecture.

Theorem 6.5 (Incompatibility diagnostic). *Assume $\mathcal{M}$ is consistent and all verifiers are sound in $\mathcal{M}$. Then no evidence ledger contains both I and either P or N .*

Proof. If I and P were both accepted, $\mathcal{M}$ would prove both $\Gamma \not\vdash c$ and $\Gamma \vdash c$, contradicting consistency. The case with N is identical. $\square$

Corollary 6.6 (Audit alarm). *If a running implementation accepts both an independence certificate and a proof certificate for the same frozen instance, then at least one of the following has occurred: a verifier is unsound, the instance was not actually frozen, the object/metatheory boundary was crossed incorrectly, or the metatheory is inconsistent.*

7 Positive certificates of nonderivability

7.1 Finite saturation certificates

Definition 7.1 (Finite saturation certificate). A finite saturation certificate for $(\mathcal{F}, \Gamma, Y)$ consists of:

- (i) a finite set $Y \subseteq \mathcal{Y}$ containing Γ and the target formulas under consideration;
- (ii) a certificate that Y is inference-closed under every rule of $\mathcal{F}$;
- (iii) a finite sequence $S_0, \dots, S_n \subseteq Y$ with $S_0 = \Gamma$ and $S_{k+1} = D_{\mathcal{F}}(S_k) \cap Y$;
- (iv) a fixed-point certificate $S_n = S_{n+1}$.

Theorem 7.2 (Finite saturation completeness). *For a valid finite saturation certificate,*

$$S_n = \text{Con}_{\mathcal{F}}(\Gamma).$$

Consequently, for every $\varphi \in Y$,

$$\varphi \notin S_n \implies \Gamma \not\vdash \varphi.$$

Proof. Because Y is inference-closed and contains Γ , every proof from Γ remains in Y . The iterative sequence is the canonical one-step closure sequence. Its fixed point contains Γ and is inference-closed, so leastness of $\text{Con}_{\mathcal{F}}(\Gamma)$ gives $\text{Con}_{\mathcal{F}}(\Gamma) \subseteq S_n$. Conversely each stage contains only consequences of Γ , by induction on k , so $S_n \subseteq \text{Con}_{\mathcal{F}}(\Gamma)$. $\square$

Corollary 7.3 (Dual saturation independence). *If a valid finite saturation certificate has $c \notin S_n$ and $\neg c \notin S_n$, then c is independent of Γ in $\mathcal{F}$.*

7.2 Invariant certificates

Definition 7.4 (Invariant exclusion certificate). An invariant exclusion certificate for a target φ consists of a decidable property P of formulas and checked evidence that:

- (i) $P(\gamma)$ holds for every $\gamma \in \Gamma$;
- (ii) every rule of $\mathcal{F}$ preserves P ;
- (iii) $P(\varphi)$ fails.

Theorem 7.5 (Invariant nonderivability). *A valid invariant exclusion certificate for φ implies $\Gamma \not\vdash \varphi$.*

Proof. Rule induction shows that every consequence of Γ satisfies P , while φ does not. $\square$

Corollary 7.6 (Dual invariant independence). *If valid invariant exclusion certificates exist for both c and $\neg c$ (possibly using different invariants), then c is independent of Γ .*

7.3 Model certificates

Definition 7.7 (Model certificate). A model certificate for a set of formulas Δ is a finite code a for a structure $\mathfrak{A}$ together with an $\mathcal{M}$ -certificate that $\mathfrak{A} \models \Delta$. The structure may be finite and directly checkable, or infinite with a finite metatheoretic proof that the coded presentation satisfies Δ .

Theorem 7.8 (Countermodel nonderivability). *Assume soundness of the proof calculus with respect to the semantics represented in $\mathcal{M}$. If a valid model certificate establishes*

$$\mathfrak{A} \models \Gamma \cup \{\neg c\},$$

then $\Gamma \not\vdash c$. If a valid model certificate establishes

$$\mathfrak{B} \models \Gamma \cup \{c\},$$

then $\Gamma \not\vdash \neg c$.

Proof. If $\Gamma \vdash c$, semantic soundness would force every model of Γ to satisfy c , contradicting $\mathfrak{A} \models \neg c$. The other direction is symmetric. $\square$

Theorem 7.9 (Model-pair independence). *A verified pair*

$$\mathfrak{A} \models \Gamma \cup \{\neg c\}, \quad \mathfrak{B} \models \Gamma \cup \{c\}$$

is an independence certificate for c over Γ .

Proof. Apply Theorem 6.7 twice. □

Theorem 7.10 (Semantic existence of model pairs). *Assume classical first-order logic with soundness and completeness. If c is independent of Γ , then there exist structures $\mathfrak{A}$ and $\mathfrak{B}$ satisfying*

$$\mathfrak{A} \models \Gamma \cup \{\neg c\}, \quad \mathfrak{B} \models \Gamma \cup \{c\}.$$

Proof. If $\Gamma \cup \{\neg c\}$ were inconsistent, the deduction theorem would yield $\Gamma \vdash c$, contrary to independence. Hence it is consistent, and completeness supplies $\mathfrak{A}$. The second model follows symmetrically. □

Remark 7.11 (Existence is not effective availability). The theorem gives semantic existence, not necessarily a finite or feasibly checkable model representation. Finite model finders can supply excellent certificates when finite models exist, but some independent theories require infinite models or sophisticated constructions.

Theorem 7.12 (NoConsequence equivalence). *Assume classical first-order logic with soundness and completeness, and let Γ be satisfiable. The following are equivalent:*

- (i) $\Gamma \not\vdash c$ and $\Gamma \not\vdash \neg c$;
- (ii) *there are models $\mathfrak{A}, \mathfrak{B} \models \Gamma$ with $\mathfrak{A} \models c$ and $\mathfrak{B} \models \neg c$.*

This is the logical content of the TPTP SZS NoConsequence status for (Γ, c) [1].

Proof. The implication (ii) $\Rightarrow$ (i) is the countermodel theorem in both directions. For (i) $\Rightarrow$ (ii), apply semantic existence of model pairs. □

7.4 Relative-consistency certificates

Definition 7.13 (Syntactic consistency certificate). A consistency certificate for a theory Δ is a metatheoretic certificate of $\Delta \not\vdash \perp$.

Theorem 7.14 (Relative-consistency independence). *Assume a standard negation calculus in which*

$$\Gamma \vdash c \implies \Gamma \cup \{\neg c\} \vdash \perp,$$

and symmetrically for $\neg c$. If $\mathcal{M}$ verifies

$$\text{Cons}(\Gamma \cup \{c\}) \quad \text{and} \quad \text{Cons}(\Gamma \cup \{\neg c\}),$$

then c is independent of Γ .

Proof. Consistency of $\Gamma \cup \{\neg c\}$ rules out $\Gamma \vdash c$. Consistency of $\Gamma \cup \{c\}$ rules out $\Gamma \vdash \neg c$. □

7.5 General metaproof certificates

Definition 7.15 (General nonderivability certificate). A general nonderivability certificate for φ is any finite object π such that the declared metatheoretic verifier checks a proof in $\mathcal{M}$ of $\Gamma \not\vdash_{\mathcal{F}} \varphi$.

Finite saturation, invariants, model certificates, proof-theoretic conservation theorems, ordinal analyses, forcing constructions, and relative-consistency arguments are all instances of this general form once their checking obligations are made explicit.

Theorem 7.16 (Paired nonderivability principle). *Any verified nonderivability certificate for c together with any verified nonderivability certificate for $\neg c$ yields a valid $\perp$ certificate, regardless of whether the two certificates use the same method.*

Proof. Conjoin the two checked metatheoretic conclusions. □

8 Certificate horizons and ALD halting

8.1 Minimal certificate length

Definition 8.1 (Status certificate length). For $e \in \{\mathbf{P}, \mathbf{N}, \mathbf{B}, \mathbf{I}\}$ define

$$\|e\|_{\mathcal{F}, \Gamma, c} := \min\{\text{len}(\pi) : \text{Ver}_e(\mathcal{F}, \Gamma, c, \pi) = 1\},$$

with value ∞ when no accepted certificate exists.

Definition 8.2 (Settlement horizon). Define the certificate settlement horizon

$$h(\mathcal{F}, \Gamma, c) := \min_{e \in \{\mathbf{P}, \mathbf{N}, \mathbf{B}, \mathbf{I}\}} \|e\|_{\mathcal{F}, \Gamma, c}.$$

Theorem 8.3 (Certificate-Horizon Theorem). *Consider a canonical ALD that enumerates every finite certificate string by nondecreasing length and applies the appropriate verifier. It halts if and only if at least one accepted status certificate exists. In that case its first halting certificate has length exactly $h(\mathcal{F}, \Gamma, c)$.*

Proof. Every certificate shorter than h is rejected by definition. At length h , at least one certificate is accepted. If no accepted certificate exists, every string is rejected and the enumeration never halts. □

Remark 8.4. The theorem generalizes the proof-horizon idea: the first decisive object need not be a proof of c . It may be a proof of $\neg c$, a contradiction certificate, or a positive certificate of independence.

8.2 Fair multiagent coverage

Definition 8.5 (Fair scheduler). A scheduler is fair for a collection of certificate-search agents if every agent that remains enabled receives infinitely many search steps, and every finite candidate generated by an enabled complete enumerator is eventually submitted for verification.

Definition 8.6 (Certificate-covered instance). An instance is certificate-covered by the agent family if at least one agent is complete for some certificate class containing a valid certificate for the instance.

Theorem 8.7 (Fair coverage halting). *A fair multiagent ALD halts on every certificate-covered instance.*

Proof. Let π be a valid certificate in a class completely enumerated by agent A_i . Fairness gives A_i infinitely many steps, so it eventually generates π . The verifier accepts it, and the configured conclusive status is reached. $\square$

Corollary 8.8 (Detectable independence). *If an independent instance admits a certificate in any class completely searched by the independence agent—for example, a finite model pair, dual invariant, finite saturation certificate, or enumerated formal metaproof—then the ALD eventually halts with INDEPENDENT rather than waiting forever for two object-theory proof searches to fail.*

8.3 Unknown safety

Theorem 8.9 (Budget exhaustion is not refutation). *Let a run stop after a finite resource budget without an accepted certificate. Sound ALD semantics permits only the result UNKNOWN (possibly annotated with the exhausted budget). It does not permit REFUTABLE, INDEPENDENT, or INCONSISTENT.*

Proof. Those conclusive statuses require evidence bits, and evidence bits enter the ledger only through accepted certificates. Resource exhaustion supplies no such certificate. $\square$

Corollary 8.10 (No false independence from dual timeout). *Two ATPs timing out respectively on c and $\neg c$ do not constitute an independence certificate, regardless of the size of the exhausted budgets.*

8.4 Undecidability barriers

Theorem 8.11 (No total sound and complete ALD for undecidable theoremhood). *Let Γ be a fixed consistent recursively axiomatized theory whose theoremhood problem is undecidable. There is no computable ALD that, for every sentence c , always halts with exactly one of*

DERIVABLE, REFUTABLE, INDEPENDENT,

and is sound and complete for these meanings:

$$\text{DERIVABLE} \iff \Gamma \vdash c, \quad \text{REFUTABLE} \iff \Gamma \vdash \neg c, \quad \text{INDEPENDENT} \iff (\Gamma \not\vdash c \wedge \Gamma \not\vdash \neg c).$$

Proof. Such an ALD would decide theoremhood: run it on c and answer yes exactly when the output is DERIVABLE. Consistency makes the three cases mutually exclusive, and completeness makes one of them occur. This contradicts undecidability. $\square$

Theorem 8.12 (No uniformly complete recursively enumerable independence certificates). *Let Γ be consistent and recursively axiomatized with undecidable theoremhood. Suppose proofs of c and proofs of $\neg c$ are recursively enumerable. There is no recursively enumerable, sound independence-certificate class that contains a certificate for every sentence satisfying*

$$\Gamma \not\vdash c \quad \text{and} \quad \Gamma \not\vdash \neg c.$$

Proof. Otherwise, on input c , dovetail three enumerations: proofs of c , proofs of $\neg c$, and independence certificates. Consistency prevents both proof enumerations from succeeding. By the assumed coverage, exactly one of the three channels eventually succeeds. Answer yes only in the first case. This decides theoremhood, a contradiction. $\square$

Remark 8.13 (What the barrier does not say). It does not make independence detection useless. It says there is no single computable certificate family complete for every sentence of an undecidable theory. Large and important subclasses can still be covered: finite propositional systems, finite algebras, theories with effective finite-model properties, instances admitting invariants, and specific deep independence results with formalized metaproofs.

Corollary 8.14 (Necessary role of unknown). *For sufficiently expressive theories, every computable sound ALD must either fail to halt on some inputs or retain a nonconclusive status such as UNKNOWN for some inputs.*

9 How the four agents help one another

9.1 Cross-role contributions

The shared pool is not merely a cache for the proponent. A verified record can change the search geometry of any role, while its support label prevents unsafe cross-branch reuse.

Contributor	Possible pool addition	Possible beneficiary
A_+	Normalization lemma, equivalence, decomposition theorem	A_- may use it to simplify $\neg c$; $A_\perp$ may combine it with an incompatible fact.
A_-	Counter-lemma or alternate normal form	A_+ may discover that its target is equivalent to a more tractable formulation.
$A_\perp$	Inconsistent core, derived contradiction premise	All agents may prune branches depending on the inconsistent core; the ALD may settle globally.
A_{ind}	Model facts, invariant lemmas, conservation facts, closure bounds	The proof agents may use object-level consequences extracted from those analyses; the inconsistency agent may test model assumptions.
Any agent	Verified intermediate object theorem	Every other object-level search can reuse it immediately in later rounds.
Any agent	Verified metatheoretic lemma	Every certificate-search method in $\mathcal{M}$ may reuse it without importing it into $\mathcal{F}$.

Theorem 9.1 (Shared-pool logical neutrality). *For a fixed frozen object theory, sound typed verifiers, and support-admissible use, the shared lemma pool cannot change which of c , $\neg c$, or $\perp$ are derivable, nor whether c is independent. It can change only the cost and order of discovering certificates.*

Proof. Base records are conservative, supported records are used only in contexts that discharge their support, and metapool facts cannot enter $\mathcal{F}$ without a checked import. Therefore logical status is unchanged. Search order and derivation length may change because accepted records become reusable. $\square$

Definition 9.2 (Pool-assisted settlement cost). For a frozen ALD implementation, seed, resource protocol, and sharing policy, let

$$E_{\text{ALD}}^{\text{pool}}(\mathcal{F}, \Gamma, c)$$

be the number of global agent expansions before the first accepted conclusive certificate, with value ∞ if none is ever emitted. Define $E_{\text{ALD}}^{\text{private}}$ analogously with private pools.

Remark 9.3. Unlike logical status, the ratio

$$\frac{E_{\text{ALD}}^{\text{private}}}{E_{\text{ALD}}^{\text{pool}}}$$

can be large, small, or less than one in a resource-limited implementation. The theory predicts soundness neutrality, not automatic performance improvement. This is an empirical target for the locked benchmark protocol.

9.2 A blackboard interpretation

The global pool is a verified, provenance-aware blackboard. The difference from an unrestricted blackboard is that publication is not trust-based: every object record carries a proof and support label, every meta-record carries a metaproof, and consumers use only records admissible in their contexts. This creates a clean trust boundary between high-variance search and low-variance checking.

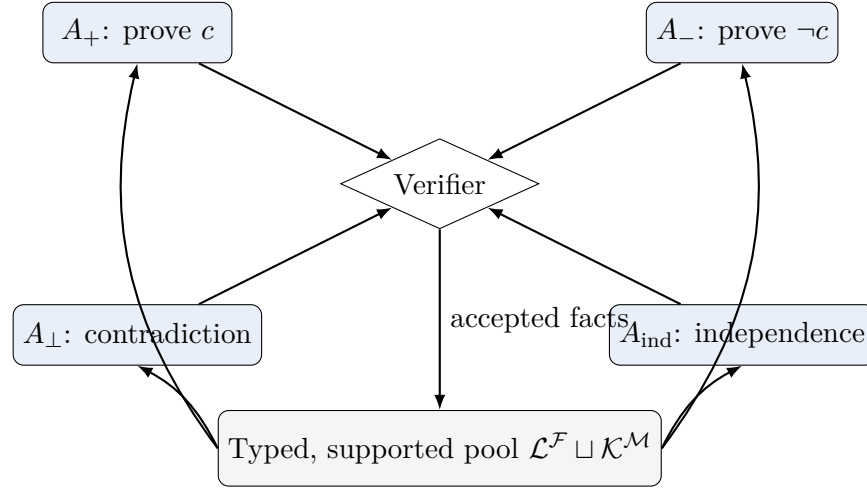


Figure 1: Every agent reads and writes through the verifier-controlled shared pool in every round.

10 Machine-learned guidance as a later layer

10.1 What learning may control

A learned policy may rank:

- which agent receives the next expansion;
- which open goal or certificate obligation is selected;
- which inference, premise, model size, invariant template, or lemma is tried next;
- which verified pool lemmas are retrieved for a local state;
- whether a candidate lies just outside a deterministic cap and should receive counterfactual exploration.

Learning changes allocation of finite search effort. It does not change certificate acceptance.

Theorem 10.1 (Verifier-separation theorem). *Fix sound certificate verifiers. Replace any deterministic or random ALD search policy by an arbitrary learned policy, even an adversarial one. Every accepted status remains sound.*

Proof. The policy determines only which candidates are proposed and in what order. Acceptance is still determined by the unchanged sound verifier. $\square$

Definition 10.2 (Fairness-preserving learned policy). Let π_{ML} be a learned scheduler and π_0 a fair baseline scheduler. A mixed policy is fairness-preserving if it reserves a recurrent positive share of search for π_0 , for example

$$\pi = (1 - \varepsilon)\pi_{\text{ML}} + \varepsilon\pi_0 \quad (0 < \varepsilon \leq 1),$$

or uses a deterministic schedule that invokes π_0 infinitely often.

Theorem 10.3 (Fair guidance preserves certificate completeness). *If the baseline agent family is complete for a certificate class and the mixed scheduler is fairness-preserving, then learned guidance preserves eventual discovery of every certificate in that class.*

Proof. The fair baseline receives infinitely many steps. Its complete enumeration therefore progresses without bound and eventually reaches any fixed finite certificate. $\square$

Remark 10.4 (Controlled creativity). The exploration component has the same role as controlled counterfactual creativity in Predator 8.002: learning pulls useful actions toward the active frontier, while a protected exploration mechanism prevents an imperfect ranking from permanently excluding them.

10.2 Pool-aware learning

The shared pool creates new learning targets. A lemma can be scored by cross-agent utility rather than only by whether it appeared in one final proof. Possible features include:

- the number and diversity of agents that reused the lemma;
- reduction in open goals or model constraints after retrieval;
- proximity to a conclusive certificate;
- logical generality, proof cost, and dependency depth;
- whether the lemma bridged object and meta search through an explicitly checked interface.

A value model may estimate the probability that a global state will yield any conclusive certificate within the remaining budget. Separate heads may estimate P, N, B, and I prospects. A meta-controller may allocate compute among agents while each local policy ranks its own actions.

Protocol 10.5 (Leakage-controlled ALD training). For a benchmark target c :

- (i) freeze the formal database, checker, agent code, and certificate formats;
- (ii) exclude the target proof, target-specific independence proof, downstream results, and manually selected target routes from training;

- (iii) train only on earlier verified traces and earlier verified pool-use events;
- (iv) preserve a fair exploration channel;
- (v) evaluate all statuses with independent fresh-process verification;
- (vi) report proof/certificate length, expansions, wall time, memory, status, and bounded unknown separately.

11 Examples of ALD settlement

11.1 Finite propositional logic

For a finite propositional formula c relative to finite assumptions Γ , the proponent and opponent may use SAT proof search. A satisfying assignment for $\Gamma \cup \{\neg c\}$ is a countermodel certificate showing $\Gamma \not\models c$; a satisfying assignment for $\Gamma \cup \{c\}$ shows $\Gamma \not\models \neg c$. If both assignments exist, the model-pair theorem certifies independence. If one side is unsatisfiable, a checked LRAT-style refutation can certify derivability of the opposite side. Here an ALD can be total because the finite truth-assignment space is decidable.

11.2 Finite algebra and first-order theories

A theorem prover can search for a derivation while a finite model finder searches for a countermodel. A model does not refute the conjecture syntactically by itself; the model certificate plus semantic soundness proves nonderivability. Two models, one satisfying c and one satisfying $\neg c$, yield independence when both are models of the same hypothesis theory.

11.3 Set-theoretic independence

For statements such as the Continuum Hypothesis, ordinary proof and refutation ATPs over ZFC are insufficient. The independence agent must search for or replay metatheoretic constructions showing, relative to an appropriate consistency assumption, a model of $\text{ZFC}+c$ and a model of $\text{ZFC}+\neg c$. The crucial computational event is not that two ZFC searches run forever. It is that a stronger formal environment checks two positive model or relative-consistency certificates.

11.4 Predator-style ALD

A Predator ALD can use four principal populations over a frozen Metamath environment:

- one searches for the target theorem;
- one searches for its formal negation;
- one searches for an explicit contradiction or incompatible pair;
- one searches a metatheoretic library for saturation, invariant, model, interpretation, or relative-consistency certificates.

At every round, all populations read and write the verified lemma pool. The Metamath checker remains the object-level authority; a separate checker validates metatheoretic certificate forms. ML may guide all populations and pool retrieval, while the checker boundary remains unchanged.

12 Further theory

12.1 Settlement depth vectors

Definition 12.1 (Status depth vector). For a fixed ALD implementation and protocol define

$$\mathbf{d}(\mathcal{F}, \Gamma, c) = (d_+, d_-, d_\perp, d_{\text{ind}}),$$

where d_e is the least global round at which an e -certificate is accepted, or ∞ if none is ever accepted.

This vector retains more information than a single final status. It distinguishes a conjecture proved quickly but contradicted later from one proved quickly in a theory for which no inconsistency evidence is found.

Proposition 12.2 (Contradiction depth bound). *Assume a fixed rule derives $\perp$ from c and $\neg c$ in at most k additional global expansions after both proofs are available. Then*

$$d_\perp \leq \max(d_+, d_-) + k$$

whenever both d_+ and d_- are finite, unless a direct contradiction is found earlier.

Proof. At round $\max(d_+, d_-)$ both proof certificates are available in the pool. The contradiction agent can combine them using the fixed derivation within k expansions. $\square$

12.2 Certificate transport under recoding

Theorem 12.3 (Recoding invariance of ALD status). *Let t be an invertible syntactic recoding transporting $\mathcal{F}$, Γ , c , proofs, models, and verifier inputs to a recoded instance. Assume every certificate verifier is equivariant under t . Then the original and recoded instances have the same accepted evidence types and the same logical status.*

Proof. Transport accepted certificates forward with t and backward with t^{-1} . Equivariance preserves acceptance, so each evidence type exists on one side exactly when it exists on the other. $\square$

Remark 12.4. A recoding may change implementation cost if the parser, index, or learned features are presentation-sensitive, but it cannot change the certificate-defined status.

12.3 Settlement as a partially ordered evidence process

The evidence ledger grows monotonically, but display statuses need not form a simple linear progression: DERIVABLE can later become BOTH, while INDEPENDENT is incompatible with either under soundness. The more fundamental state is therefore the set E_r , ordered by inclusion. Logical settlement is an evidence accumulation process, and the display status is a projection of that richer state.

Proposition 12.5 (Persistent evidence). *Once an evidence bit enters E_r , it remains in every E_n for $n \geq r$.*

Proof. The ledger update is by union. $\square$

Corollary 12.6 (Persistent certified claims). *A later search failure, timeout, or policy change cannot revoke an already accepted certificate. Revocation requires discovering that the verifier, frozen environment, or certificate was faulty, which is an audit event outside the formal run.*

12.4 Research conjectures

Conjecture 12.7 (Cross-status lemma advantage). There are natural benchmark families for which a universally shared verified lemma pool yields an unbounded asymptotic reduction in settlement expansions over every architecture with role-isolated pools, even though both architectures use the same local ATP algorithms.

Conjecture 12.8 (Certificate-diversity advantage). For some natural families, no single certificate method is efficient on all instances, while a fair ALD combining proof, refutation, model, invariant, and saturation agents has strictly better worst-case finite-budget coverage than any one constituent method.

Conjecture 12.9 (Learned pool utility transfer). A leakage-controlled model trained to predict cross-agent lemma utility transfers better to unfamiliar theorem domains than a model trained only to predict membership in final proofs, because cross-agent reuse supplies a denser and more task-general supervision signal.

Conjecture 12.10 (ALD reflective ceiling). For sufficiently expressive sound metatheories, there is no single internally verified ALD that both decides every object-theory sentence and fully certifies the soundness of all of its own status verifiers. Any practical hierarchy of ALDs therefore has a reflective ceiling or requires an external trust anchor.

13 Implementation blueprint

Listing 1: Auditable round-synchronous ALD skeleton

```

input: frozen object theory F, hypotheses Gamma, conjecture c
initialize private agent states Q[i]
initialize provenance-aware typed pools L_object, L_meta
initialize evidence ledger E = empty

for round r = 1, 2, 3, ...:
    snapshot = (L_object, L_meta, E)

    proposals = parallel_for each agent i:
        return agent_step(i, Q[i], admissible_view(i, snapshot))

    for proposal p in deterministic_audit_order(proposals):
        if p.type == OBJECT_LEMMA and verify_object_with_support(p):
            L_object.add(p)
        elif p.type == META_LEMMA and verify_meta(p):
            L_meta.add(p)
        elif p.type in {PROOF_C, PROOF_NOT_C, PROOF_BOTTOM,
                        INDEPENDENCE_CERT} and verify_status(p):
            E.add(status_evidence_type(p))

    write_checkpoint(r, Q, L_object, L_meta, E)

    if frozen_halt_policy(Status(E)):
        emit(Status(E), accepted_certificates(E), audit_log)
        halt

    if frozen_resource_budget_exhausted():
        emit(UNKNOWN, budget_record, audit_log)

```

halt

13.1 Required audit record

A reproducible ALD run should record:

- hashes of the object theory, metatheory, checker binaries, agent code, and ML models;
- the conjecture, negation encoding, contradiction encoding, and hypotheses;
- the random seed and deterministic proposal-merge order;
- every accepted and rejected status certificate;
- every lemma-pool insertion, its logical level, assumption support, origin agent, proof certificate, support promotions, and first-use events;
- per-agent expansions, wall time, memory, and checkpoint state;
- the exact halting policy and resource budget;
- fresh-process verification of the final certificate bundle.

14 Limitations and boundaries

- L1. Independence is metatheory-relative.** A certificate proves what $\mathcal{M}$ can soundly establish about $\mathcal{F}$.
- L2. No uniform totality.** Undecidability prevents a computable sound and complete ALD from settling every sentence of sufficiently expressive theories.
- L3. Model certificates may be unavailable or enormous.** Semantic existence does not imply a finite practical artifact.
- L4. Finite saturation requires true closure.** A fixed point inside an arbitrary truncated search space proves only bounded exhaustion, not global nonderivability.
- L5. Invariants are sufficient, not necessary.** Failure to find an invariant is UNKNOWN.
- L6. The pool can hurt performance.** Soundness survives irrelevant lemmas, but memory, provenance checks, and ranking can degrade; relevance filtering and support minimization are engineering necessities.
- L7. Learning does not certify.** It proposes and prioritizes; only the verifier licenses settlement.
- L8. A contradiction changes interpretation.** In an inconsistent object theory, deriving c does not exclude deriving $\neg c$. The evidence ledger is more informative than a single truth-like label.

15 Conclusion

The central problem of computer-detected independence is not solved by waiting for two proof searches to fail. It is solved, when it is solvable, by searching for positive certificates that both derivations are impossible. A multiagent ALD turns this into an explicit architecture: proof, negation-proof, inconsistency, and independence roles run concurrently; every agent can add to and inspect a shared verified lemma pool in every round; object and meta records remain distinct; and assumption provenance prevents a branch theorem from being mistaken for a theorem of the base theory. Finite saturation, invariants, model pairs, relative consistency, and formal metaproofs become first-class halting mechanisms. Timeouts remain bounded unknown. Machine learning may later allocate search attention and predict cross-agent lemma utility without weakening soundness, provided certificate checking, provenance enforcement, and fairness remain outside the learned component.

The literature already contains cooperative provers, blackboard systems, lemma exchange, model-based non-theorem statuses, the SZS *NoConsequence* relation, formal independence proofs, and learned proof guidance. The research opportunity is therefore not to claim those pieces individually. It is to develop and test their unified certificate-indexed theory: a SIC-style ALD with a monotone evidence ledger and a universally visible, provenance-aware lemma pool. Undecidability prevents this from becoming a universal total decision procedure, but it does not prevent many formerly silent non-halting cases from becoming finitely, positively, and audibly settled.

References and research context

References

- [1] G. Sutcliffe, “The SZS Ontologies and Presentation Standards,” TPTP World documentation, including the *NoConsequence* status and proof/model data forms, accessed August 2, 2026.
- [2] M. P. Bonacina and J. Hsiang, “The Clause-Diffusion Methodology for Distributed Deduction,” *Fundamenta Informaticae*, 24(1–2):177–207, 1995. DOI: 10.3233/FI-1995-24128.
- [3] J. Denzinger, M. Kronenburg, and S. Schulz, “DISCOUNT—A Distributed and Learning Equational Prover,” *Journal of Automated Reasoning*, 18(2):189–198, 1997. DOI: 10.1023/A:1005879229581.
- [4] D. Fuchs, “Requirement-Based Cooperative Theorem Proving,” in *Logics in Artificial Intelligence: JELIA ’98*, LNAI 1489, pp. 139–153, Springer, 1998.
- [5] C. Benz Müller, N. Sultana, L. C. Paulson, and F. Theiß, “The Higher-Order Prover Leo-II,” *Journal of Automated Reasoning*, 55:389–404, 2015. DOI: 10.1007/s10817-015-9348-y.
- [6] M. Wisniewski and C. Benz Müller, “Is It Reasonable to Employ Agents in Automated Theorem Proving?” in *Proceedings of ICAART 2016*, vol. 1, pp. 281–286. DOI: 10.5220/0005824702810286.
- [7] D. Kim, “Distributed Agent-Based Automated Theorem Proving in Order-Sorted First-Order Logic,” *arXiv:1609.02320*, 2016.
- [8] J. Harrison and L. Théry, “A Skeptic’s Approach to Combining HOL and Maple,” *Journal of Automated Reasoning*, 21:279–294, 1998. DOI: 10.1023/A:1006023127567.
- [9] W. McCune, “Mace4 Reference Manual and Guide,” *arXiv:cs/0310055*, 2003.
- [10] G. Reger, M. Suda, and A. Voronkov, “Finding Finite Models in Multi-Sorted First-Order Logic,” *arXiv:1604.08040*, 2016.
- [11] J. M. Han and F. van Doorn, “A Formal Proof of the Independence of the Continuum Hypothesis,” *arXiv:2102.02901*, 2021.
- [12] C. Kaliszyk and J. Urban, “Learning-Assisted Theorem Proving with Millions of Lemmas,” *Journal of Symbolic Computation*, 69:109–128, 2015. DOI: 10.1016/j.jsc.2014.09.032.
- [13] C. Kaliszyk, J. Urban, and J. Vyskočil, “Lemmatization for Stronger Reasoning in Large Theories,” in *FroCoS 2015*, LNAI 9322, pp. 341–356. DOI: 10.1007/978-3-319-24246-0_21.
- [14] J. Jakubův and J. Urban, “ENIGMA: Efficient Learning-Based Inference Guiding Machine,” *arXiv:1701.06532*, 2017.
- [15] K. Baba, C. Liu, S. Kurita, and A. Sannai, “Prover Agent: An Agent-Based Framework for Formal Mathematical Proofs,” *arXiv:2506.19923*, 2025.
- [16] C. E. Brown, C. Kaliszyk, and J. Urban, “Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents,” *arXiv:2603.06737*, 2026.